

TRABAJO PRÁCTICO 7: HERENCIA Y POLIMORFISMO EN JAVA



Resolución Caso Práctico:

1. Vehículos y herencia básica

Paso 1: Crear la clase Vehículo

Se creó la clase Vehiculo con los atributos marca y modelo, un constructor para inicializarlos y el método mostrarInfo() que imprime la información del vehículo.

Código:

```
public class Vehiculo {  
    private String marca;  
    private String modelo;  
  
    public Vehiculo(String marca, String modelo) {  
        this.marca = marca;  
        this.modelo = modelo;  
    }  
  
    public void mostrarInfo() {  
        System.out.println("Marca: " + marca);  
        System.out.println("Modelo: " + modelo);  
    }  
}
```

Paso 2: Crear la clase Auto

Se creó la subclase Auto que hereda de Vehiculo, agrega el atributo cantidadPuertas y sobrescribe el método mostrarInfo() para incluir la cantidad de puertas.

Código:

```
public class Auto extends Vehiculo {  
    private int cantidadPuertas;  
  
    public Auto(String marca, String modelo, int cantidadPuertas) {  
        super(marca, modelo);  
        this.cantidadPuertas = cantidadPuertas;  
    }  
}
```

```

    }

    @Override
    public void mostrarInfo() {
        super.mostrarInfo();
        System.out.println("Cantidad de puertas: " + cantidadPuertas);
    }
}

```

Paso 3: Instanciar un Auto y mostrar su información

Se instanció un objeto miAuto de la clase Auto y se llamó al método mostrarInfo() para imprimir todos sus datos.

Código:

```

public class Main {
    public static void main(String[] args) {
        Auto miAuto = new Auto("Toyota", "Corolla", 4);
        miAuto.mostrarInfo();
    }
}

```

Captura de la ejecución en consola:

```

run:
Marca: Toyota
Modelo: Corolla
Cantidad de puertas: 4
BUILD SUCCESSFUL (total time: 0 seconds)

```

2. Figuras geométricas y métodos abstractos

Paso 1: Crear la clase abstracta Figura

Se creó la clase abstracta Figura con el atributo nombre, un método abstracto calcularArea() y un método mostrarInfo() que imprime el nombre y área de la figura.

Código:

```

public abstract class Figura {
    protected String nombre;
}

```

```

public Figura(String nombre) {
    this.nombre = nombre;
}

public abstract double calcularArea();

public void mostrarInfo() {
    System.out.println("Figura: " + nombre);
    System.out.println("Área: " + calcularArea());
}
}

```

Paso 2: Crear las subclases Circulo y Rectangulo

Se crearon las subclases Circulo y Rectangulo que implementan el método abstracto calcularArea() de Figura. Cada una calcula su área específica según su geometría.

Código:

```

public class Circulo extends Figura {
    private double radio;

    public Circulo(double radio) {
        super("Círculo");
        this.radio = radio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }
}

public class Rectangulo extends Figura {
    private double ancho;
    private double alto;
}

```

```

public Rectangulo(double ancho, double alto) {
    super("Rectángulo");
    this.ancho = ancho;
    this.alto = alto;
}

@Override
public double calcularArea() {
    return ancho * alto;
}
}

```

Paso 3: Crear el array de figuras y mostrar sus áreas

Se creó un array de Figura con diferentes objetos (Circulo y Rectangulo). Se recorrió el array y se llamó a mostrarInfo() de cada figura, demostrando polimorfismo.

Código:

```

public class Main {
    public static void main(String[] args) {
        Figura[] figuras = new Figura[3];
        figuras[0] = new Circulo(5);
        figuras[1] = new Rectangulo(4, 6);
        figuras[2] = new Circulo(3);

        for (Figura f : figuras) {
            f.mostrarInfo();
            System.out.println("-----");
        }
    }
}

```

Captura de la ejecución en consola:

```
Figura: Círculo
Área: 78.53981633974483
-----
Figura: Rectángulo
Área: 24.0
-----
Figura: Círculo
Área: 28.274333882308138
-----
BUILD SUCCESSFUL (total time: 0 seconds)
```

3. Empleados y polimorfismo

Paso 1: Crear la clase abstracta Empleado

Se creó la clase abstracta Empleado con el atributo nombre, un método abstracto calcularSueldo() y un método mostrarInfo() para imprimir el nombre y sueldo del empleado.

Código:

```
public abstract class Empleado {
    protected String nombre;

    public Empleado(String nombre) {
        this.nombre = nombre;
    }

    public abstract double calcularSueldo();

    public void mostrarInfo() {
        System.out.println("Empleado: " + nombre);
        System.out.println("Sueldo: " + calcularSueldo());
    }
}
```

Paso 2: Crear las subclases EmpleadoPlanta y EmpleadoTemporal

Se crearon las subclases EmpleadoPlanta y EmpleadoTemporal que implementan el método calcularSueldo() de manera específica según el tipo de empleado.

Código:

```
public class EmpleadoPlanta extends Empleado {
    private double salarioFijo;

    public EmpleadoPlanta(String nombre, double salarioFijo) {
        super(nombre);
        this.salarioFijo = salarioFijo;
    }

    @Override
    public double calcularSueldo() {
        return salarioFijo;
    }
}

public class EmpleadoTemporal extends Empleado {
    private double horasTrabajadas;
    private double valorHora;

    public EmpleadoTemporal(String nombre, double horasTrabajadas, double valorHora) {
        super(nombre);
        this.horasTrabajadas = horasTrabajadas;
        this.valorHora = valorHora;
    }

    @Override
    public double calcularSueldo() {
        return horasTrabajadas * valorHora;
    }
}
```

Paso 3: Crear lista de empleados y usar polimorfismo e instanceof

Se creó una lista de Empleado con objetos de EmpleadoPlanta y EmpleadoTemporal. Se recorrió la lista, se llamó a calcularSueldo() polimórficamente y se usó instanceof para clasificar a cada empleado.

Código:

```
import java.util.ArrayList;

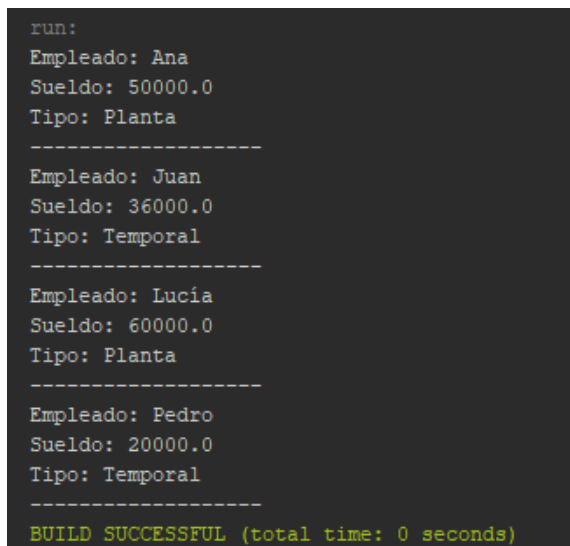
public class Main {
    public static void main(String[] args) {
        ArrayList<Empleado> empleados = new ArrayList<>();
        empleados.add(new EmpleadoPlanta("Ana", 50000));
        empleados.add(new EmpleadoTemporal("Juan", 120, 300));
        empleados.add(new EmpleadoPlanta("Lucía", 60000));
        empleados.add(new EmpleadoTemporal("Pedro", 80, 250));

        for (Empleado e : empleados) {
            e.mostrarInfo();

            if (e instanceof EmpleadoPlanta) {
                System.out.println("Tipo: Planta");
            } else if (e instanceof EmpleadoTemporal) {
                System.out.println("Tipo: Temporal");
            }

            System.out.println("-----");
        }
    }
}
```

Captura de la ejecución en consola:



```
run:
Empleado: Ana
Sueldo: 50000.0
Tipo: Planta
-----
Empleado: Juan
Sueldo: 36000.0
Tipo: Temporal
-----
Empleado: Lucía
Sueldo: 60000.0
Tipo: Planta
-----
Empleado: Pedro
Sueldo: 20000.0
Tipo: Temporal
-----
BUILD SUCCESSFUL (total time: 0 seconds)
```


4. Animales y comportamiento sobrescrito

Paso 1: Crear la clase Animal

Se creó la clase Animal con el atributo nombre, un método hacerSonido() genérico y describirAnimal() que imprime el nombre del animal.

Código:

```
public class Animal {  
    protected String nombre;  
  
    public Animal(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public void hacerSonido() {  
        System.out.println("El animal hace un sonido genérico");  
    }  
  
    public void describirAnimal() {  
        System.out.println("Animal: " + nombre);  
    }  
}
```

Paso 2: Crear las subclases Perro, Gato y Vaca

Se crearon las subclases Perro, Gato y Vaca, cada una sobrescribiendo el método hacerSonido() para mostrar su sonido característico.

Código:

```
public class Perro extends Animal {  
    public Perro(String nombre) {  
        super(nombre);  
    }  
  
    @Override  
    public void hacerSonido() {  
        System.out.println("Guau guau");  
    }  
}
```

```

public class Gato extends Animal {
    public Gato(String nombre) {
        super(nombre);
    }

    @Override
    public void hacerSonido() {
        System.out.println("Miau miau");
    }
}

```

```

public class Vaca extends Animal {
    public Vaca(String nombre) {
        super(nombre);
    }

    @Override
    public void hacerSonido() {
        System.out.println("Muuuu");
    }
}

```

Paso 3: Crear lista de animales y mostrar sus sonidos

Se creó una lista de Animal con objetos Perro, Gato y Vaca. Se recorrió la lista y se llamó a hacerSonido() polimórficamente, mostrando el sonido característico de cada animal.

Código:

```

import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Animal> animales = new ArrayList<>();
        animales.add(new Perro("Rex"));
        animales.add(new Gato("Michi"));
        animales.add(new Vaca("Lola"));
    }
}

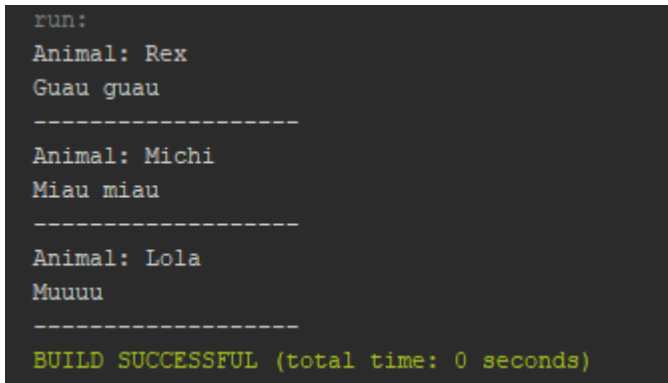
```

```

        for (Animal a : animales) {
            a.describirAnimal();
            a.hacerSonido();
            System.out.println("-----");
        }
    }
}

```

Captura de la ejecución en consola:



```

run:
Animal: Rex
Guau guau
-----
Animal: Michi
Miau miau
-----
Animal: Lola
Muuuu
-----
BUILD SUCCESSFUL (total time: 0 seconds)

```

5. Sistema de pagos con polimorfismo y genéricos

Paso 1: Crear la interfaz Pagable

Se creó la interfaz Pagable con el método pagar(double monto) que será implementado por distintos tipos de pago.

Código:

```

public interface Pagable {
    void pagar(double monto);
}

```

Paso 2: Crear las clases que implementan Pagable

Se crearon las clases TarjetaCredito, Transferencia y Efectivo, cada una implementando la interfaz Pagable con su versión del método pagar().

Código:

```

public class TarjetaCredito implements Pagable {
    private String titular;

    public TarjetaCredito(String titular) {
        this.titular = titular;
    }
}

```

```

    }

    @Override
    public void pagar(double monto) {
        System.out.println("Pagando " + monto + " con tarjeta de crédito de " + titular);
    }
}

public class Transferencia implements Pagable {
    private String cuenta;

    public Transferencia(String cuenta) {
        this.cuenta = cuenta;
    }

    @Override
    public void pagar(double monto) {
        System.out.println("Pagando " + monto + " mediante transferencia a la cuenta " + cuenta);
    }
}

public class Efectivo implements Pagable {
    @Override
    public void pagar(double monto) {
        System.out.println("Pagando " + monto + " en efectivo");
    }
}
}

```

Paso 3: Crear función procesarPago() y probar distintos medios de pago

Se creó la función `procesarPago(Pagable medio, double monto)` que permite procesar cualquier tipo de pago. Se probaron distintos medios (TarjetaCredito, Transferencia y Efectivo) demostrando polimorfismo.

Código:

```

public class Main {
    public static void procesarPago(Pagable medio, double monto) {

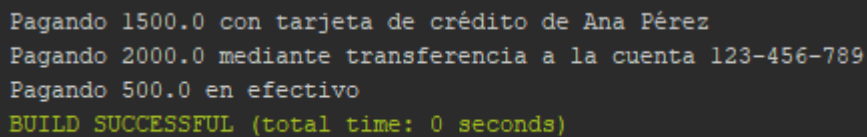
```

```
        medio.pagar(monto);
    }

    public static void main(String[] args) {
        Pagable pago1 = new TarjetaCredito("Ana Pérez");
        Pagable pago2 = new Transferencia("123-456-789");
        Pagable pago3 = new Efectivo();

        procesarPago(pago1, 1500);
        procesarPago(pago2, 2000);
        procesarPago(pago3, 500);
    }
}
```

Captura de la ejecución en consola:



```
Pagando 1500.0 con tarjeta de crédito de Ana Pérez
Pagando 2000.0 mediante transferencia a la cuenta 123-456-789
Pagando 500.0 en efectivo
BUILD SUCCESSFUL (total time: 0 seconds)
```

Link a mi repositorio de GitHub:

<https://github.com/NazarenoAranda/UTN-TUPaD-P2-Nazareno-Aranda/tree/master/UTN-TUPaD-P2-Nazareno-Aranda>

Nazareno Aranda